



LIONSDEV

Curso de Programação

MÓDULO 12

Projeto Final e Requisitos



Professor Nicolas Cardoso Motta

Revisando o Módulo 11



Telas

Widgets (**Column**, **Row**, **Container**), estado e navegação entre páginas.



API Call

A tela chama a rota, mapeia o JSON e mostra na lista; o token vai no header.



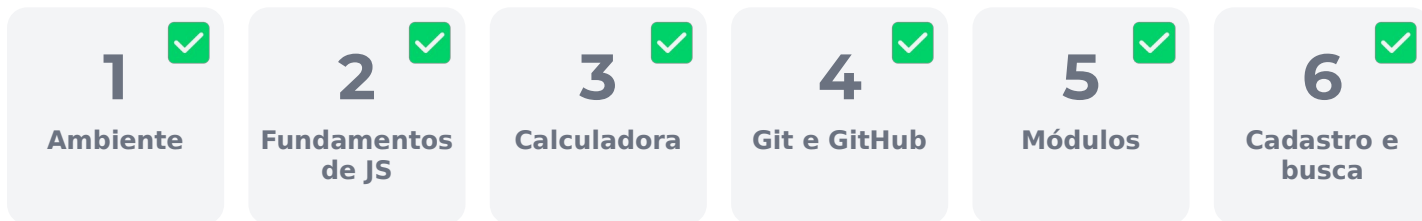
Quatro estados

Carregando, vazio, erro e sucesso: toda tela com API precisa dos quatro.

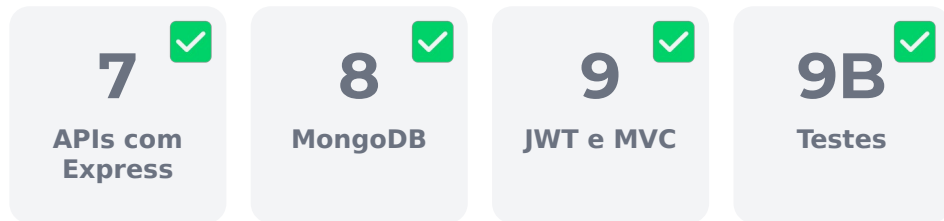
Pergunta para abrir: qual foi a parte mais difícil de ligar a tela na API?

Mapa do curso

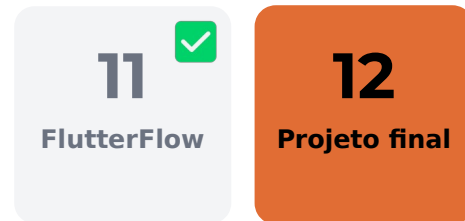
BASE: JAVASCRIPT, GIT E ORGANIZAÇÃO



BACKEND



FRONTEND E PROJETO



Você está aqui: **Módulo 12**, Projeto Final e Requisitos



Objetivo

- **Descobrir o problema de um cliente por meio de uma entrevista.**
- **Transformar a conversa em requisitos, escopo e entidades.**
- **Planejar rotas, telas e a divisão do trabalho no grupo.**
- **Construir, publicar e apresentar uma aplicação full stack.**
- **Saber exatamente como o projeto vai ser avaliado.**

**Do Módulo 1 ao 11, você aprendeu
cada peça. Agora a pergunta muda:
não é mais "como eu faço isso?", e sim
"o que eu devo construir?"**

O que você vai entregar



Backend

API REST com Node.js, Express, MongoDB e Mongoose, com autenticação e senhas protegidas.



Frontend

Telas feitas com IA ou FlutterFlow, consumindo a API de verdade.



Deploy

Backend e frontend publicados no Render, conversando entre si.



Documentação

Levantamento de requisitos, README, coleção do Postman e registro do uso de IA.

O foco não é uma interface perfeita: é um sistema que funciona de ponta a ponta.

Regras do jogo



Grupos de 2 a 3

Todos participam da apresentação e explicam a própria parte.



Tema do grupo

Resolve um problema claro, cabe no prazo e não repete o tema de outro grupo.



Prazo

Definido em aula. O levantamento de requisitos vem antes do código.



Foco

Simples e funcional vale mais que complexo e quebrado. Mas precisa parecer um sistema real, não um CRUD solto.

Do problema à apresentação



Descobrir

Entrevista
com o cliente.

Planejar

Requisitos,
escopo e
entidades.

Construir

Backend e
frontend.

Publicar

Deploy no
Render.

Apresentar

Demonstração
ao vivo.

Não pule etapas: código escrito antes do planejamento costuma ir para o lixo.



Simulação cliente-dev

Descobrir o problema antes de escrever código

O cliente quase nunca sabe dizer o que quer em termos técnicos. Mas ele sabe muito bem o que o **incomoda.
Descobrir isso é trabalho do dev.**

Dois papéis



Cliente

- Fala como uma pessoa comum, sem termos técnicos.
- Tem um problema, não uma solução pronta.
- Não conta tudo de uma vez: responde o que perguntarem.
- Mantém a história coerente do começo ao fim.



Dev

- Pergunta, confirma e organiza as respostas.
- Não lê a página do cliente.
- Não propõe solução técnica cedo demais.
- No fim, preenche o levantamento de requisitos.

Depois, troquem os papéis e rodem de novo, com outro projeto.

Cliente: monte a sua história



Quem é você?

Ex.: dono de uma pequena cafeteria.



Qual dor te incomoda hoje?

Ex.: perco o controle das encomendas que anoto num caderno.



Como você resolve hoje, sem sistema?

Ex.: anoto tudo e, às vezes, esqueço de avisar o cliente.



O que você gostaria que melhorasse?

Ex.: não perder pedido e avisar quando ficar pronto.



Quem mais usaria?

Ex.: a funcionária que atende o balcão.

Se travar, use um cenário



Cafeteria

Perde o controle das encomendas anotadas no caderno.



Professor

Tem várias turmas e se enrola com entregas e notas de trabalhos.



Personal trainer

Monta treinos para os alunos e perde o histórico no papel.



Eventos

Controla convidados e confirmações por mensagens soltas.

Dev: técnicas do entrevistador



Perguntas abertas primeiro

"Me conta como você faz isso hoje." Sim ou não fica para o final.



Pergunte o porquê

Pediu uma solução pronta? "Por que você precisa disso?"



Palavra vaga pede exemplo

Rápido, fácil, simples, seguro: "rápido quanto?"



Confirme repetindo

"Então, o que você precisa é..."



Essencial x "seria legal"

Separe o que resolve a dor do que é enfeite.

Pergunta fechada x aberta

Fechada

Dev: "Você quer um app de encomendas?"

Cliente: "Quero."

Nada foi aprendido.

Aberta

Dev: "Me conta como você controla as encomendas hoje?"

Cliente: "Anoto no caderno, mas às vezes a folha some e eu esqueço de avisar quando fica pronto..."

Apareceram o processo, o problema e uma regra.

Roteiro da entrevista

Bloco	Uma pergunta para começar
1. Contexto atual	Como você faz isso hoje, sem nenhum sistema?
2. Problema	Se você pudesse resolver uma coisa só, qual seria?
3. Quem usa	Além de você, quem mais usaria?
4. O que espera	Se o sistema já existisse, o que você faria primeiro?
5. Restrições	Tem alguma regra, prazo ou limite que eu preciso saber?
6. Prioridades	Se fossem só três coisas primeiro, quais seriam?

Use como guia, não como formulário: a conversa pode mudar de ordem.



Cuidado: o cliente pede a solução, não o problema

"Quero um app igual ao iFood." Isso é uma **solução. O problema real pode ser só não perder encomendas.**

- Solução pronta esconde o problema: pergunte "por que isso te ajudaria?" até chegar na dor.
- Palavra vaga (rápido, fácil, seguro) precisa virar exemplo ou número.
- Não traduza para termos técnicos na frente do cliente: anote as palavras dele.

Atividade Prática

Entrevista

1. Formem duplas: um é o cliente, o outro é o dev.
2. O cliente monta a história (5 minutos), sem mostrar ao dev.
3. O dev entrevista usando o roteiro e anota as respostas (15 minutos).
4. O dev resume em voz alta: "então, o que você precisa é...", e o cliente corrige.
5. Troquem os papéis, com outro projeto.



Dica: o dev anota as palavras do cliente, não a tradução técnica. "Perco o caderno" é mais útil que "precisa de persistência".



Desafio: no fim, o dev lista as três prioridades do cliente, e o cliente confere.

Checklist antes de encerrar



Problema

Consigo descrevê-lo com as palavras do cliente.



Usuários

Sei quem vai usar o sistema.



Escopo

Sei o que está dentro e o que fica fora.



Essencial

Tenho os requisitos que não podem faltar.



Regras

Sei as regras e os limites do negócio.



Prioridade

Sei o que é mais urgente para o cliente.



Levantamento de requisitos

Da conversa para o documento

Da entrevista para o template

Bloco da entrevista	Vira no levantamento
Contexto atual	Público-alvo e problema
Problema / dor	Problema, solução proposta e objetivos
Quem usa	Público-alvo e atores
O que espera	Escopo, telas, endpoints, entidades e requisitos funcionais
Restrições e limites	Fora do escopo, requisitos não funcionais e regras de negócio
Prioridades	A coluna de prioridade dos requisitos

O template está em `modulo12/lista_de_exercicios/levantamento_requisitos.md`.

Exemplo: a cafeteria

Vamos seguir o mesmo exemplo até o fim do módulo.



Público-alvo

A dona da cafeteria e a funcionária do balcão.



Problema

As encomendas ficam num caderno: pedidos se perdem e o cliente não é avisado quando fica pronto.



Solução

Um sistema onde o balcão registra as encomendas, acompanha o status e vê o que está pronto.

Escopo: o que entra e o que fica fora



O sistema fará

- Cadastro e login.
- Produtos e categorias.
- Encomendas com itens, ligadas a um cliente.
- Status da encomenda até "entregue".



O sistema não fará

- Pagamento online.
- Aviso por WhatsApp.
- App para o cliente final.

O "fora do escopo" protege o prazo. Tudo o que está no levantamento aparece no sistema, ou está aqui.

Requisitos funcionais e não funcionais



Funcionais: o que faz

- **RF-003:** o sistema deve permitir registrar uma encomenda com um ou mais produtos.
- **RF-004:** o sistema deve permitir mudar o status da encomenda até "entregue".



Não funcionais: como se comporta

- **RNF-001:** as senhas devem ser guardadas com hash.
- **RNF-002:** o backend deve ser separado em model, repository, service, controller e routes.

Frases específicas e testáveis, cada uma com um ID e uma prioridade.



Cuidado: requisito que não dá para testar

"O sistema deve ser rápido e fácil de usar." Ninguém consegue conferir se isso foi cumprido.

- Escreva algo que dê para testar no Postman ou na tela.
- Troque a palavra vaga por um comportamento ou por um número.

Ruim: O sistema deve ser fácil de usar.

Bom: RF-005: listar as encomendas do dia, por horário de retirada.

Ruim: O cadastro deve ser rápido.

Bom: RNF-003: cadastrar uma encomenda em menos de 30 segundos.

Regras de negócio

RN-001

Um usuário só vê e altera os próprios registros.

RN-002

Uma encomenda só pode ser marcada como entregue se estiver pronta.

RN-003

Produto inativo não entra em encomenda nova.

É o que diferencia o seu sistema de um CRUD genérico. O projeto precisa de pelo menos uma regra que envolva mais de uma entidade, como a RN-003.

Priorizando

Prioridade	Significado
Essencial	Sem isso, o sistema não funciona. Tem que ser feito.
Importante	Funciona sem isso, mas fica incompleto. Faça se houver tempo.
Desejável	Não compromete o funcionamento. Pode ficar para uma próxima versão.

Primeiro, todos os essenciais funcionando. Só depois os importantes.



Modelando o sistema

Entidades, campos e relacionamentos

O cliente fala, o dev traduz



O que ele guarda vira entidade

Coisas que precisam ser lembradas.

```
"anoto as encomendas de cada cliente"  
→ Encomenda, Cliente
```



O que ele faz vira rota e tela

Ações sobre essas coisas.

```
"marco quando fica pronto"  
→ PATCH /api/encomendas/:id/status
```

Cliente quase nunca fala "minha entidade é Pedido". Quem faz essa tradução é o dev.

Entidades por tema

Tema	Entidades possíveis
Cafeteria	Produto, Categoria, Cliente, Pedido, ItemPedido
Academia	Aluno, Plano, Matrícula, Treino, Avaliação
Eventos	Evento, Lote, Participante, Inscrição, Check-in
Biblioteca	Livro, Autor, Categoria, Empréstimo, Reserva
Clínica	Paciente, Profissional, Especialidade, Consulta, Prontuário

Exemplos para inspirar: o tema e as entidades são do grupo.

As entidades da cafeteria

Entidade	Principais campos	Relaciona-se com
Categoria	nome	tem vários Produtos
Produto	nome, preço, ativo	pertence a uma Categoria
Cliente	nome, telefone	tem várias Encomendas
Encomenda	itens, status, data de retirada	um Cliente; itens apontam para Produtos
Usuario (boilerplate)	nome, e-mail, senhaHash	dono dos registros

Quatro entidades do domínio além do Usuario: o mínimo exigido no projeto.

O model da Encomenda

```
const { ObjectId } = mongoose.Schema.Types
const STATUS = ['recebida', 'em preparo', 'pronta', 'entregue']

const encomendaSchema = new mongoose.Schema(
  {
    cliente: { type: ObjectId, ref: 'Cliente', required: true },
    itens: [{
      produto: { type: ObjectId, ref: 'Produto', required: true },
      quantidade: { type: Number, required: true, min: 1 },
    }],
    status: { type: String, enum: STATUS, default: 'recebida' },
    dono: { type: ObjectId, ref: 'Usuario', required: true },
  },
  { timestamps: true }
)
```

Cliente e produto por referência (ref); os itens dentro da encomenda; o dono vem do token, como os livros do Módulo 9.

Embedding ou referência?



Embedding: dentro

Os itens só existem dentro da encomenda: não faz sentido buscar um item sozinho.



Referência: o _id

Cliente e produto têm vida própria: aparecem em várias encomendas e têm o próprio CRUD.

Na dúvida: esse dado é consultado ou editado sozinho? Então ele é uma entidade, ligada por referência.



Cuidado: o Usuario não conta

O **Usuario** do boilerplate existe para o **login**. Ele não conta como entidade do domínio.

- São pelo menos 4 entidades além do Usuario, com pelo menos 1 relacionamento entre elas.
- Quatro CRUDs isolados não formam um sistema: precisa de uma regra que ligue as entidades.
- Se cada usuário tem os próprios dados, as entidades guardam o dono e filtram por **req.usuario.id**.



Planejando o backend

Boilerplate, rotas e regras de negócio

O mínimo do backend



Dados

- 4 entidades do domínio, além do Usuario.
- 2 entidades com CRUD completo.
- 1 relacionamento por referência.
- 1 regra de negócio entre entidades.



Código

- Camadas: model, repository, service, controller e routes.
- Middleware de tratamento de erro.
- Validação dos campos obrigatórios.
- Rotas das entidades com o middleware **autenticar**.

Dados fixos em array não são aceitos: tudo salvo, buscado, editado e removido no MongoDB Atlas.

Partindo do boilerplate

```
$ git clone https://github.com/nicolassmotta/boilerplate-lions-dev.git cafeteria-api  
$ cd cafeteria-api  
$ git remote set-url origin https://github.com/grupo/cafeteria-api.git  
$ npm install  
$ cp .env.example .env  
$ npm start
```

Crie antes o repositório **vazio** do grupo no GitHub. Depois do cp, preencha o MONGO_URI e o JWT_SECRET no .env.

Um recurso novo, arquivo por arquivo

```
src/  
├── models/encomenda.model.js  
├── repositories/encomenda.repository.js  
├── services/encomenda.service.js  
├── controllers/encomenda.controller.js  
└── routes/encomenda.routes.js  
  
// no app.js  
app.use("/api/encomendas", encomendaRoutes)
```

repository → o único que conversa com o banco.

service → onde moram as regras de negócio.

controller → recebe o **req**, chama o **service** e responde.

routes → declara os endpoints e aplica os middlewares.

As rotas de um CRUD

Rota	O que faz	Sucesso
POST /api/encomendas	Cadastra	201
GET /api/encomendas	Lista	200
GET /api/encomendas/:id	Busca por ID	200
PATCH /api/encomendas/:id	Atualiza	200
DELETE /api/encomendas/:id	Exclui	200 ou 204

Todas protegidas: `router.post("/", autenticar, EncomendaController.criar)`

Além do CRUD: rotas de fluxo

Uma rota que conecta entidades e aplica uma regra: é o que faz o projeto parecer um sistema.

Adicionar um item

POST /api/pedidos/:id/itens

Concluir uma consulta

PATCH /api/consultas/:id/concluir

Emprestar um livro

POST /api/emprestimos

Cancelar uma matrícula

PATCH /api/matriculas/:id/cancelar

O projeto precisa de pelo menos uma rota assim.

A regra de negócio mora no service

```
// services/encomenda.service.js
async function registrar(idDoUsuario, { cliente, itens }) {
  if (!itens || itens.length === 0) {
    throw criarErro('A encomenda precisa de pelo menos um item.', 400)
  }
  for (const item of itens) {
    const produto = await ProdutoRepository.buscarPorId(item.produto)
    if (!produto || !produto.ativo) {
      throw criarErro('Produto indisponível.', 400)
    }
  }
  return EncomendaRepository.criar({ cliente, itens, dono: idDoUsuario })
}
```

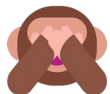
A RN-003 envolve duas entidades: Produto e Encomenda. O controller só chama o service; o erro com status sobe até o middleware de erro.

Regras do Módulo 9



O dono vem do token

Sempre `req.usuario.id`, nunca o `req.body`. Registro de outra pessoa: 404.



A senha nunca volta

Nem a senha, nem o `senhaHash`, em nenhuma resposta.



Segredos no .env

O `.env` fica fora do GitHub; o `.env.example` vai, sem valores reais.



Frontend, deploy e testes

A API consumida de verdade, e no ar

O mínimo do frontend



Login

Consumindo **POST**
`/api/auth/login`.



Listagem

Dados vindos do
backend.



Cadastro

Formulários para 2
entidades.



Editar ou excluir

Em pelo menos 2
entidades.



Relações

Uma tela com dados de
entidades ligadas.



Mensagens

Sucesso e erro na tela.

Os quatro estados do Módulo 11 contam: carregando, vazio, erro e sucesso.

IA ou FlutterFlow?



Frontend com IA

- React (ou outro) gerado com apoio de IA.
- Publicado como Static Site no Render.



FlutterFlow

- Telas montadas visualmente, com API Calls.
- Web Publishing, ou o código publicado no Render.

Qualquer caminho vale, desde que consuma a API real. O grupo é responsável por entender, testar e ajustar tudo o que foi gerado.

Registro do uso de IA

Uso de IA

Ferramenta: Claude, ChatGPT, FlutterFlow...

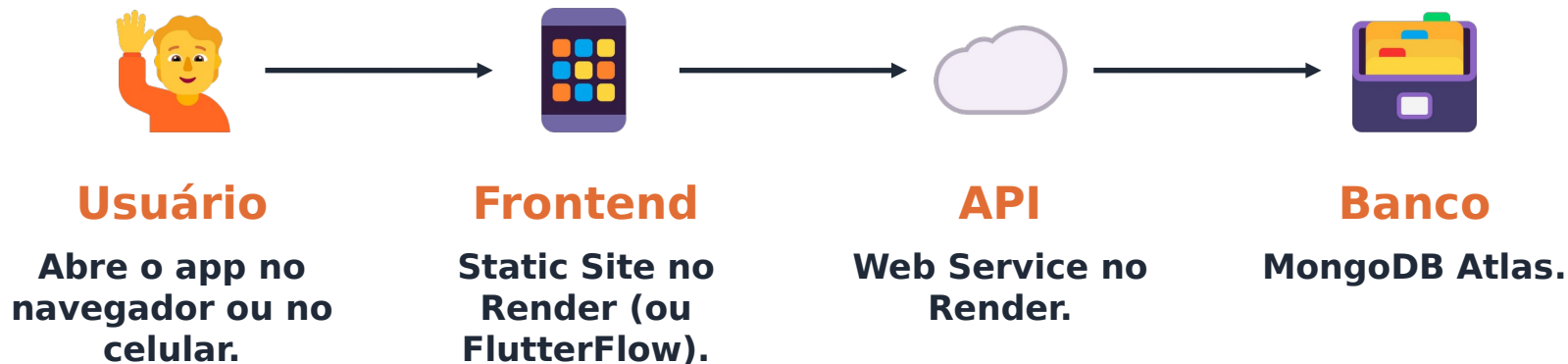
O que foi gerado: tela de login e listagem de encomendas

O que ajustamos: nomes dos campos (preço, e não price);
o header Authorization nas rotas protegidas

Erros e solução: 401 na listagem → o token não ia no header

Obrigatório no README. Na apresentação, o grupo precisa explicar o que a IA gerou e o que foi corrigido.

A arquitetura do projeto



Os dois links, do frontend e do backend, vão no README.

Deploy no Render



Backend: Web Service

Build: `npm install`

Start: `npm start`

Variáveis no painel:

`MONGO_URI`, `JWT_SECRET`,
`JWT_EXPIRES_IN`,
`BCRYPT_SALT_ROUNDS`, `NODE_ENV`



Frontend: Static Site

React/Vite

Build: `npm install && npm run build`

Publish: `dist`

FlutterFlow

Publish: `build/web`

Rewrite: `tudo para index.html`

O backend usa `process.env.PORT`, como no boilerplate. O `.env` não sobe: as variáveis ficam no painel.



Cuidado: o front publicado chamando localhost

No seu computador funciona. Para o resto do mundo, `localhost` é a máquina de **cada um: nada carrega.**

- A versão publicada usa a URL do backend no Render, em `https`.
- A API libera a origem do front no CORS, senão o navegador bloqueia.
- Plano free: o primeiro acesso pode demorar (cold start). Abra o backend antes de apresentar.

```
app.use(cors({ origin: 'https://cafeteria-front.onrender.com' })))
```

O que testar no Postman

Caminho feliz: deve dar certo	Tente quebrar: deve dar erro
Cadastro e login	Sem token → 401
Listar e buscar por ID	Token inválido → 401
Criar (201)	Campo obrigatório vazio → 400
Atualizar (200)	ID inexistente → 404
Excluir (200 ou 204)	ID mal formatado → 400
Rota da regra de negócio	Regra violada → 400

Teste na URL local e na URL do Render. O frontend não substitui os testes no Postman.

A coleção do Postman



Uma coleção por projeto

Com uma pasta por recurso: Auth, Produtos, Encomendas...



Variáveis

Crie as variáveis **baseUr1** (local ou Render) e **token**: troque de ambiente sem reescrever as requisições.



Exporte e versione

Exporte a coleção (formato v2.1) e coloque o arquivo **.json** no repositório.



Entrega e avaliação

README, equipe, apresentação e nota

O README do projeto

```
# Nome do Projeto
## Descrição
## Tecnologias
## Funcionalidades
## Como rodar
## Variáveis de ambiente
## Rotas da API
## Frontend
## Deploy
## Postman
## Uso de IA
## Integrantes
```

Como rodar → `npm install`, criar o `.env`, `npm start`.

Variáveis → o que cada uma faz, sem valores reais.

Deploy → os links do backend e do frontend no Render.

Uso de IA → o registro do slide anterior.



Cuidado: o .env foi para o GitHub

Apagar o **.env** num commit novo **não resolve**: ele continua no histórico do Git.

- Troque a senha do usuário do banco no Atlas e gere um novo **JWT_SECRET**: o segredo antigo virou público.
- Previna: **.env** no **.gitignore** antes do primeiro commit, e só o **.env.example** no repositório.

```
$ echo ".env" >> .gitignore
$ git rm --cached .env      # tira do Git, sem apagar do computador
$ git commit -m "remove o .env do repositório"
```

Divisão de trabalho



Integrante 1

Backend de parte das entidades.



Integrante 2

Frontend e integração com a API.



Integrante 3

Requisitos, README e Postman.



Todos

Testes no Postman e apresentação.

Cada um é dono de pelo menos uma funcionalidade. Uma branch por funcionalidade e Pull Request para a main (Módulo 4).

Cada um precisa saber explicar



O que fez

A funcionalidade que ficou sob a sua responsabilidade.



Quais arquivos alterou

O histórico de commits mostra quem fez o quê.



Qual rota ou tela é sua

E como ela conversa com o resto do sistema.



Quais erros encontrou

E como resolveu: é aqui que se vê quem aprendeu.

Quem não contribuiu ou não sabe explicar a própria parte pode receber nota individual menor.

Roteiro da apresentação

- 1 Problema e público-alvo**
Para quem é e qual dor resolve.
- 2 Requisitos e modelagem**
Entidades, relacionamentos e a organização do backend.
- 3 Demonstração ao vivo**
Login, CRUD e a regra de negócio entre entidades.
- 4 Os dados no Atlas**
Mostre que o que foi criado na tela está no banco.
- 5 No ar**
Frontend e backend publicados no Render.
- 6 IA, dificuldades e próximos passos**
O que ajudou, o que travou e o que viria depois.

Como o projeto vai ser avaliado

Critério	O que conta	Peso
Backend / API	CRUD completo, camadas, 4+ entidades, regra entre entidades	20%
Autenticação	Senha com hash, login com JWT, rotas protegidas	15%
Frontend	Telas consumindo a API: login, listagem, cadastro, mensagens	15%
Banco de dados	Persistência real no Atlas: salva, busca, edita e exclui	10%
Deploy	Backend e frontend no Render, integrados de ponta a ponta	10%
Requisitos	Coerente com o tema, e aparece no sistema	10%
Organização	GitHub, README, .env.example e coleção do Postman	10%
Uso de IA	Documentado; o grupo explica o que gerou e ajustou	5%
Apresentação	Clareza, demonstração ao vivo e todos participando	5%

Cada critério vale de 0 a 10, e a nota final é a média ponderada. A nota individual pode ser ajustada.

Checklist final

Funcionando

- Backend e front sem erro.
- MongoDB salvando de verdade.

No ar

- Back e front no Render.
- Front usando a URL pública da API.

Backend

- 4+ entidades além do Usuario.
- 2 CRUDs, 1 relacionamento, 1 regra.

Segurança

- Hash, JWT e rotas com autenticar.
- .env fora, .env.example dentro.

Documentação

- Requisitos e README com os links.
- Postman e uso de IA no repositório.

Equipe

- Todos sabem explicar o projeto.
- Cada um mostra a própria parte.

Atividade Prática

Kickoff

1. Formem os grupos (2 a 3) e escolham o tema, sem repetir o de outro grupo.
2. Rodem a simulação cliente-dev e preencham o levantamento de requisitos.
3. Definam as entidades, os relacionamentos e as rotas principais.
4. Criem o repositório a partir do boilerplate e dividam as tarefas.
5. Mostrem o levantamento ao professor antes de começar a programar.



Dica: comecem pelo fluxo mínimo de ponta a ponta: login, uma listagem e um cadastro funcionando no Render. Depois, cresçam.

Materiais do módulo



Enunciado do projeto final

Escopo, tecnologias, README, deploy e checklist: `projeto_final.md`.



Simulação cliente-dev

O roteiro completo da entrevista: `simulacao_cliente_dev.md`.



Template de requisitos

Para preencher antes de programar: `levantamento_requisitos.md`.



Lista de código: projeto integrado

Revisão de todas as camadas: `codigo_projeto_integrado.md`.

No repositório: `modulo12/lista_de_exercicios/`

Material do módulo



Aponte a câmera do celular

`github.com/nicolassmotta/curso-lions-dev/
tree/main/modulo12`



conteudo/
teoria comentada e o cheat sheet do módulo



lista_de_exercicios/
listas para praticar em casa



Fim!

Agora é com vocês. Bom projeto!